Java basics, definition of algorithms and basic programming model

File: S1.1-N001_basic-programming-model.md

Authors: Robert Sedgewick, Kevin Wayne

Source: Algorithms, 4th

Location: ch. 1, sec. 1.1, pp. 1-40

Links:

- https://hai.stanford.edu/ai-definitions/what-is-an-algorithm
- https://algs4.cs.princeton.edu/11model/

What is an Algorithm

An **Algorithm** is a set of instructions for solving a problem or complete a task. They tell how to process some data input and manipulate it to reach out some solution for determined problem that we can possibly have. The term *algorithm* can be described by a finite, deterministic, and effective problem-solving method. It is something that is indepedent of any language, and mainly concerns about the steps, the method that is used to solve some problem, and we can translate that in computer science using programming languages. For example, the following statement says:

> Compute the greatest common divisor of two nonnegative integers p and q as follows: If q is 0, the answer is p. If not, divide p by q and take the remainder r. The answer is the greatest common divisor of q and r.

We've used English here, but in Java, we can do the same statement as follows:

```
public static int gcd(int p, int q) {
    if (q == 0) return p;
    int r = p % q;
    return gcd(q, r);
}
```

This is the *Euclid's algorithm*. Keep in mind that we have so many ways to solve some problem using a computer, and hardly matters which approach we'll use, with the exception of large-scale application where we need to solve a large amount of small problemas and the impact of each one counts.

When developing a huge or complex computer program, a large part of the effort must go understanding and defining the problem to be solved, managing its complexity and breaking into down small pieces that can be implemented easily. Often, many of the algorithms, once they've been broken down, their implementation is almost trivial. As long as I'm already familiar with Java, I'll skip some concepts about the language, but here's an important information about primitive datatypes and expressions in Java: the key property of these primitive operations is that

- *An operation involving values of a given type has a value of that type.* This rule highlights the idea that we are often working with approximate values, since it is often the case that the exact value that would seem to be defined by the expression is not a value of the type.
- Casting an `int` in Java is **truncation** and not rounding. Rules for casting within complicated expressions can be intricate, and casts should be used sparingly and with care.
- Java is said to be a strongly typed language, because the Java compiler checks for consistency (for example, it does not permit us to multiply a `boolean` and a `double`).

Tipical array-processing code fragments

You can see the full explation about arrays at pp. 18-21. Here, I'll just put some useful code fragments that I can probably will need to use later.

Find the maximum of array values

```
double max = [0];
for (int i = 0; i < a.length; i++)
    if (a[i] > max) max = a[i];
```

Compute the average of array values

```
int N = a.length
for(int i = 0; i < N; i++)
    sum += a[i]
double average = sum / N;
```

Reverse the elements within an array

```
int N = a.length;
for (int i = 0; i < N/2; i++)
{
    double temp = a[i];
    a[i] = a[N-1-i];
    a[N-i-1] = temp;
}
```

Matrix-Matrix multiplication (square matrices)

Given the code `a[][]*b[][] = c[][]`

```
int N = a.length;
double[][] c = new double[N][N];
```

```
for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
    { // Compute dot product of row i and column j.
        for (int k = 0; k < N; k++)
            c[i][j] += a[i][k]*b[k][j];
    }
```

For further explanations about Java syntax, the book provides a more complete reference, covering topics such as primitive data types, strings, output formatting, and other language fundamentals. I'll refer to the book whenever I need to review these concepts, keeping these notes focused on the algorithms themselves.

Redirection and piping

We can redirect the standard output of a programing using a simple directive in command line that allows us the write and redirect the output into a text file.

```
$ java RandomSeq 1000 100.0 200.0 > data.txt
```

This command specifies that the standard output stream is not to be printed in the terminal, but instead is to be written to a text file named data.txt. Each call to `StdOut.print()` or `StdOut.println()` appends text at the end of that file.

In a similar way, we can redirect some input so that `StdIn` can read its data inseated of the data that came from terminal input.

```
$ java Average < data.txt
```

This command reads the data in `data.txt` and computers their average value using the function defined bellow (see p. 39). Specifically, the symbol `<` is a directive that tells the operating system to implement the standard input stream by reading the data present in the text file `data.txt` instead of waiting for some user's input value in the terminal window.

```java
public class Average
{
    double sum = 0.0;
    int cnt = 0;
    while(!StdIn.isEmpty())
    {
        sum += StdIn.readDouble();
        cnt++;
    }
    double average = sum / cnt;
    StdOut.printf("The average is %.5f\n", average);
}
```

Output:

```
$ java Average
1.23456
2.34567
3.45678
4.56789
<ctrl-d>
Average is 2.90123
```

When the program calls `StdIn.readDouble()`, the operating system reads the value from the file. Combining these to redirect the output of one program to the input of another is known as piping:

```
$ java RandomSeq 1000 100.0 200.0 | java Average
```

This command basically says: *"get the output returned from RandomSeq and use it as the input for the Average implementation".*